

Pré-projeto: Backlog de jogos

Instituto Hardware

1 Identificação da Equipe

- **Nome do Projeto:** SavePoint (provisório)
- **Membros e GitHub:**
 - **Victor Alexandre Silva Perim** — GitHub: [@Victor Perim](#)
 - **Adriano Pereira Silva** — GitHub: [@Adriano Pereira](#)
- **Divisão Preliminar de Responsabilidades:**
 - **Adriano:** Configuração do ambiente Flask com integração do PostgreSQL, modelagem do banco de dados relacional e controle das rotas das runs de roguelikes, saves/builds e categorias.
 - **Victor:** Estruturação de templates responsivos com Bootstrap 5, modelagem do banco de dados relacional, implementação de lógica interativa dinâmica no frontend usando JavaScript com requisições assíncronas (fetch) e rotas do backlog de jogos, conquistas e amizades.

2 Elevator Pitch

Para jogadores dedicados que se perdem no meio de dezenas de jogos acumulados e builds complexas, o SavePoint é uma aplicação web que centraliza o gerenciamento de backlogs, histórico de runs e anotações de progresso. Diferente de usar blocos de notas espalhados ou planilhas complexas, nós oferecemos uma interface visual unificada, focada no registro manual rápido e na personalização da experiência do jogador.

3 Público-Alvo e Contexto

- **Quem vai usar?** Jogadores casuais de PC e console, entusiastas de jogos de RPG, roguelikes, speedrunners e colecionadores de conquistas/troféus.
- **Em que situação?** No monitor secundário enquanto joga na tela principal, ou no celular deitado no sofá logo após finalizar uma sessão de jogo rápida.
- **Por que isso é melhor que planilha/papel/WhatsApp?** Planilhas dão trabalho para formatar e carecem de apelo visual gamer. Blocos de notas e chats de WhatsApp perdem o histórico facilmente e não compilam estatísticas de progresso de forma automatizada e integrada.

4 Funcionalidades Principais

Para que o MVP (Produto Mínimo Viável) seja considerado concluído, o sistema deve implementar as seguintes funcionalidades:

1. Como jogador, quero cadastrar jogos em minha biblioteca com status (Quero Jogar, Jogando, Zerado, Platinado, Abandonado) e nota pessoal para organizar meu backlog de forma visual.
2. Como jogador, quero registrar o progresso de conquistas de um jogo específico (total de troféus e conquistados) para acompanhar minha porcentagem de conclusão.

3. Como jogador de roguelikes/speedruns, quero registrar minhas tentativas (runs) com tempo total, resultado (vitória/derrota) e causa da morte para analisar meu histórico de evolução.
4. Como jogador de RPG, quero criar anotações detalhadas de builds (itens equipados, distribuição de skills e talentos) para não perder o progresso estratégico entre sessões longas de jogo.
5. Como usuário, quero ver um painel geral (dashboard) com o número total de jogos zerados, tempo acumulado jogado e média de notas para visualizar as estatísticas de meu perfil.

5 Escopo Negativo — O que NÃO faremos

Para garantir a entrega dentro do prazo estipulado, delimitamos que o projeto **não** conterá:

- Integração automática com APIs de plataformas de terceiros (Steam, PlayStation Network, Xbox Live) para puxar conquistas ou tempo de jogo; todo o preenchimento será puramente manual.
- Sistemas complexos de autenticação externa (OAuth2 como Google ou Steam); usaremos um fluxo simplificado de cadastro e login baseado em sessões do Flask.
- Aplicativo mobile nativo para iOS ou Android; a solução será inteiramente focada em Web Responsiva via Bootstrap e componentes adaptáveis.
- Mecânicas de fórum público amplo ou chat em tempo real; as interações entre jogadores serão limitadas exclusivamente a amizades simples e visualização mútua de perfis.

6 Modelo de Dados Preliminar

Para expandir o potencial e a organização do sistema, incluímos entidades de Categorias/Marcadores personalizados e o sistema de Amizades:

USUARIO (id, nome, username, email, senha)
JOGO (id, usuario_id [FK], titulo, status, nota, tempo_jogado, total_conquistas, conquistas_obtidas)
CATEGORIA (id, usuario_id [FK], nome, cor_hex)
JOGO_CATEGORIA (jogo_id [FK], categoria_id [FK])
RUN_DIARIO (id, jogo_id [FK], data, tempo_duracao, resultado, causa_morte)
BUILD_ANOTACAO (id, jogo_id [FK], nome_build, detalhes Equipamento, habilidades)
AMIZADE (usuario_solicitante_id [FK], usuario_receptor_id [FK], status_solicitacao)

Relacionamentos:

- Um *Usuario* tem muitos *Jogos* ($1 \rightarrow N$).
- Um *Usuario* pode criar suas próprias *Categorias* personalizadas ($1 \rightarrow N$).
- Um *Jogo* pode ter muitas *Categorias* e uma *Categoria* pode pertencer a vários *Jogos* (Relação muitos-para-muitos $N \rightarrow M$ através de *Jogo_Categoria*).
- Um *Jogo* pode ter muitas tentativas em *Run_Diario* ($1 \rightarrow N$).
- Um *Jogo* pode conter muitas anotações em *Build_Anotacao* ($1 \rightarrow N$).
- Usuários podem se conectar via *Amizade* através de uma relação autorreferencial muitos-para-muitos.

7 Wireframes (Planejamento Visual)

Abaixo estão os esboços estruturais das principais interfaces da aplicação:

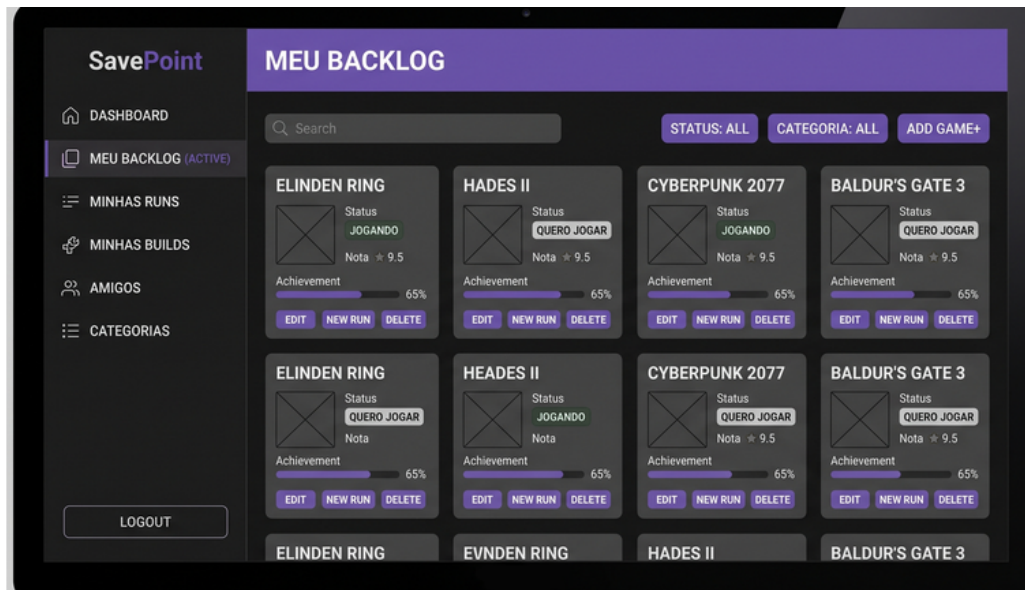


Figura 1: Interface Desktop: Visualização do Backlog em Grid.



Figura 2: Interface Mobile: Formulário de Registro Rápido de Runs.

8 Stack Confirmada

- [X] **Flask + Jinja2:** Utilizado para estruturar o roteamento de endpoints, a lógica de negócio do servidor no padrão MVC e renderizar dinamicamente os dados nas páginas HTML.
- [X] **PostgreSQL:** Banco de dados relacional robusto e escalável, suporte a consultas relacionais complexas, além de ter mais familiaridade com membros da equipe.
- [X] **HTML + Bootstrap 5 + JavaScript:** Utilização do framework Bootstrap 5 para acelerar a estilização responsiva de componentes de interface e JS para requisições assíncronas (AJAX via Fetch API) ao adicionar dados em tempo real nas tabelas sem recarregar a tela.
- [X] **GitHub com PRs:** Uso de Pull Requests obrigatórios antes de fazer merge na branch `main` para garantir a revisão de código em dupla.

9 Autoavaliação de Riscos

1. Qual é a parte que vocês mais temem implementar?

A lógica assíncrona com JavaScript integrada com as rotas do Flask para atualizar as estatísticas do dashboard e adicionar runs sem dar refresh na página, além de garantir o mapeamento correto da relação muitos-para-muitos ($N \rightarrow M$) das categorias no PostgreSQL.

2. O que acontece se um membro da equipe sumir 1 semana antes da entrega?

Como o projeto foi planejado de maneira altamente modular, as tarefas podem ser consolidadas rapidamente. O membro remanescente focará no “caminho feliz” central, desativando os recursos de Amizades e Categorias para garantir a entrega das rotas básicas estáveis.

3. Qual é o “caminho feliz” mínimo que entrega valor mesmo se tudo mais falhar?

O funcionamento do fluxo básico do Backlog de Jogos (Funcionalidade 1). Conseguir cadastrar jogos, alterar o status e atribuir notas de forma persistente no PostgreSQL já entrega o valor principal de organização proposto pelo projeto.